Whole-Route Planning of a Memory Bus Between Two BGAs: An Exact Model of Crossings and Layer Changes, Negotiated to DRC-Clean Copper

Andrew Haas

***Abstract*—Routing the bus between a processor and its DRAM remains largely manual in printed circuit board (PCB) design, and published algorithms optimize proxies such as crossing or layer counts. We present a router that plans every lane of a two-layer BGA-to-BGA bus before any copper is laid, and carries the plan to design-rule-clean copper. Its core is an exact constraint-programming model that chooses, for all lanes at once, where each necessary crossing occurs, in what order—by a rule over every triple of lanes—and where each lane changes layer, with via room; for the ends it is given, it proves its plan optimal within the model. The model reads naturally as physics: worldlines, frustrated Ising spins and contact forces. A linear program places the lanes in millimeters; the constraints it must violate return to the model as cuts, and audit findings as negotiated congestion prices, until the plan passes an audit against the production router's rules or lays clean. The escapes are chosen by an estimate of the same objective, and differential pairs carry their polarity, with an automatic via crossover where the ends disagree. On an Allwinner H3 and a Zynq-7020, each to a DDR3 device, the router completes every rung of two ladders, of up to 51 nets on the H3 (its whole bus) and 43 on the Zynq, connected and DRC-clean, never with more vias than the human designer (80 against 88 and 62 against 100 at the top rungs) and always with less copper, where the toolkit's production router leaves nets open on four of the ten rungs and Freerouting completes none. On the Zynq board itself it completes 45 of the bus's nets with 70 vias to the designer's 105. On generated buses of three, four and six routing layers every run completes, and every one with a known optimum attains it.**

***Index Terms*—Printed circuit board, bus routing, escape routing, topological via minimization, layer assignment, constraint programming, linear programming, differential pairs, DDR.**

I. Introduction

A memory bus is the densest wiring on most processor boards: tens of nets leave a ball grid array (BGA) in a fixed order, arrive at another in a different order, and must cross one another on the way. On a two-layer region every crossing forces one of the two nets onto the other layer there, and every layer change is a via—a drill, an annular ring that blocks both layers, and a discontinuity in the signal path. Commercial tools approach it interactively: the designer sequences the bus, plans its breakouts and resolves its crossings, and the tool fills in the route [2], [3].

The literature treats the problem in stages—escape routing, simultaneous escape routing, bus planning and layer assignment (Section II)—each optimizing a proxy such as crossings, layers or a planned via count, evaluated on private industrial cases. Whether a plan can be laid in copper at the board's clearance, and how its via count compares with a designer's, are left open.

This paper starts from an observation that designers, commercial tools and simultaneous escape routing share [2], [4]: *the ends decide the braid*. Once each net's escape at the source and destination is fixed, the orders of those escapes around the two arrays fix which pairs of nets must cross. What remains free is where each crossing happens, in what order the crossings occur, and where each lane changes layer. These choices interact, so we decide all of them at once, for every lane, before anything is routed. The contributions are:

1) An exact constraint-programming (CP) model of the whole route (Section IV) that chooses crossing positions, their order, and every lane's layer changes jointly. Its consistency rule over triples of lanes admits exactly the orders that a wiring diagram of the lanes' permutation can realize. For the ends it is given, it proves its plan optimal within the model.
2) A realization of the plan in board geometry (Section VI): an elastic linear program whose violated constraints return to the CP model as cuts, a snap onto the production router's grid, and an audit against the router's own rules, closed in a negotiation loop that also feeds findings at the ends back to the choice of escapes (Section V).
3) Differential pairs planned as single lanes with a polarity (handedness) at each end, and an automatic via crossover that swaps P and N where the two ends disagree.
4) An evaluation on two public boards, on benches cut from them and on one board itself, against the human layouts, the toolkit's production router and Freerouting, all graded alike on finished copper, and on generated buses of two to six layers against their exact optimum (Section VII).

The models read naturally in physics. Along the route each lane is the worldline of a particle in one space and one time dimension, each crossing a scattering event, and the triple rule the Yang–Baxter relation of three strands. Each worldline carries an Ising spin, its layer, opposite its partner's at every crossing: a via is a domain wall, and frustration forces vias. Placing the lanes is then a problem in statics, whose contact forces are the linear program's dual variables.

A. Haas is a Professor in the Department of Physics, New York University, New York, NY 10003 USA (e-mail: andy.haas@nyu.edu).

The software, the benches and the recipes that reproduce every number in this paper are open source [1]; the results are those of its commit `2e319092`.

II. Related Work

Topological via minimization. Given the nets' topology, choosing layers to minimize vias is classical. Hsu [5] decoupled topology from geometry to minimize vias; Marek-Sadowska [6] showed the unconstrained problem NP-complete, and Sarrafzadeh and Lee [7] gave a polynomial algorithm for nets between two shores, where the problem reduces to covering a permutation graph. Constrained via minimization assigns layers to an already routed layout [8]: it is NP-complete in general [9], and where the layout's junctions have degree at most three it is a maximum cut on a planar graph, solvable in polynomial time [8], [10], which Barahona *et al.* solved with the same code as Ising spin-glass ground states [11]. Exact algorithms followed for k layers in channels [12], [13] and for a package's via-free redistribution layers [14]. None of these places the crossings, orders them, or gives vias their room, jointly with the layers.

PCB escape and bus routing. Ozdal and Wong chose escape patterns at both ends of a channel jointly, with a longest-path search for each layer's planar subset and negotiation for the rest [4], and Ozdal, Wong and Honsinger minimized the vias the choice causes directly [15]. Bus planning and layer assignment for dense boards followed [16], [17]; where a bus's pin orders at its two ends disagree, Yan and Wong untangle the twisted nets with single detours near the ends, on one layer and with no vias [18], [19]. Exact models route the escapes at one array [20], [21] or a two-layer BGA package over a restricted route structure [22], and differential pairs have escape routers of their own [23]. Wu *et al.* unified escape and area routing in one model, searching escape orders by simulated annealing [24]. Lin *et al.* carried simultaneous escape routing through gridless area routing to finish commercial boards that a commercial router could not [25]. Apart from that flow and the escape routers, this work stops at a plan, and all of it is evaluated on private cases.

Topology, then geometry. Rubber-band routers [26] and the theory of homotopic routing [27] separate a topological route from its embedding. Wen *et al.* assign a package's layers by a maximum planar subset and finish its layout by linear programming [28]. Negotiated congestion [29] prices places that were overused before.

Wiring diagrams. The orders in which the crossings of a permutation's reduced words can occur are the convex orders of its inversion set [30]; for the longest permutation their commutation classes form the higher Bruhat order $B(n, 2)$ [31]. We are not aware of a model that uses this structure to minimize vias jointly with crossing positions and via room.

III. The Problem and the Frame

A. Setting

A *bench* is a board region holding a source BGA S (the processor) and a destination BGA D (the DRAM), a set of bus nets between them, and everything else—passives, other nets' stubs, the board edge—as fixed obstacles. The region has two signal layers, F and B (Section VII-F extends the model to more). Each net leaves S at a *tooth*, the end of a fanout stub from its ball, and arrives at D at a *berth*; each end has a layer. The fanout engine offers every ball a menu of escape moves (a stub on one layer, or a dog-bone via to the other), from which the ends are chosen (Section V). A differential pair is one *lane* with two legs. A lane's layer change is a *dive*: one via for a single lane, two barrels side by side for a pair. The benches' clearance is 0.1 mm; we plan and route 5 μm above it, at 0.105 mm, with 0.127 mm lanes, 0.1 mm fanout stubs and 0.25 mm vias (0.15 mm drill) on the router's grid $g = 0.025$ mm. Two neighboring lanes on one layer must lie at least a *bar* of 0.232 mm apart, center to center (a lane and the clearance); the least pitch at which lanes are planned is $p = 0.252$ mm, a bar and 20 μm.

The objective counts planned vias: one for every layer change of a lane, and five more for every via a net carries beyond two, its stub vias included (a pair counted by its worse leg); the stub vias themselves come with the ends and cost nothing more. A net whose vias are all layer changes thus costs 0, 1, 2, 8 or 14 for 0 to 4 vias. Below a via comes a congestion tie-break. These counts are of the plan, not the board: the stub vias come from the choice of ends (Section V), and a pair's layer change counts once although it lays two barrels. The result must be routed by the production router, connected, and clean under a design-rule check (DRC) at the board's clearance.

B. The frame and the route coordinate

The frame is read off the board alone (Fig. 1). The board is first turned by an exact quarter turn, and mirrored for pairs of the other handedness (both undone on writing), so that S lies west of D. Each array's pad box is grown by the median distance its stubs end outside it. S's *far* face is the one away from D; D's *facing* face is the one toward S, and its far face the one away. A straight *trunk* runs from the center of S's box to that of D's, extended past each by half its diagonal and $L = 0.35$ mm; a point near it is (s, o), arclength along it and offset across it.

The orders are unrolled from *cuts* on the far faces. On D's far face the cut stands in the middle of the widest gap between its berths, the face's ends counted as gap ends, so that berths together near one end go round one side. On S's far face it stands in the gap that makes the fewest crossings. Unrolling the berths round D from its cut, north first, gives the *berth order* φ; unrolling the teeth round S gives the *launch order* λ. The ends model hands its cuts to the frame with the plan, so that model and solve share one order.

A lane berthing on D's facing face ends in the trunk; the least s of these berths is the *arrival line* H_0. The others ride a *ring*, north or south of D's cut. They leave the trunk at the ring's start H_k, stepped back from H_0, stacked in berth order a lane's width apart outside the facing face's lanes and the octilinear hull of D's pads, stubs and the copper hugging it. The ring's spine runs taut from there to the hull and then round it, L out, the way its lanes' taut paths sweep. A tooth on S's far face reaches its side face on a *lead-in*, straight out of the face and round S's corner on a 45° chamfer, nested with the others of its side; from there the trunk carries it. Each lane's *reference* is its taut path, moved out of both boxes on the lane's own side.

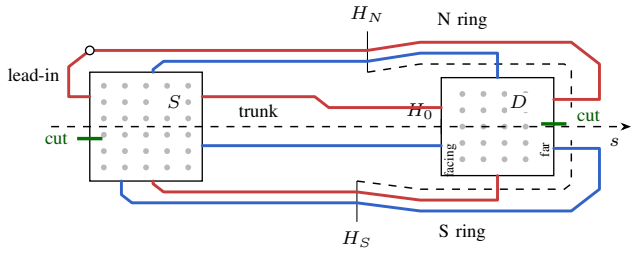


Fig. 1. The frame, schematically. The trunk (dashed, arclength s) runs through the grown boxes of S and D. Lanes berthing on D's facing face end in the trunk, the first at H_0; the others leave it at their ring's start (H_N, H_S), stacked in berth order, and ride a ring to D's side faces and to its far face, north or south of its cut (green). A tooth north of S's cut reaches the north face on a lead-in, to its start (circle). These six lanes keep one order and cross nowhere. Red is F, blue is B.

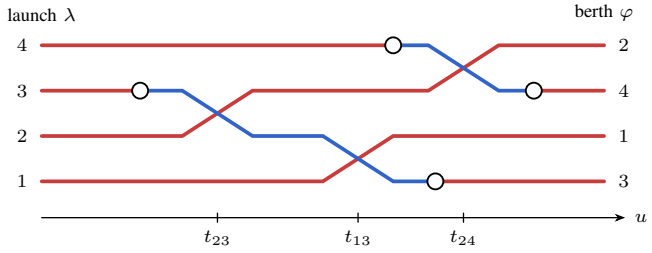


Fig. 2. Four lanes as a wiring diagram along the route coordinate u. The berth order is $\pi = (3, 1, 4, 2)$, so the inverted pairs $\{2, 3\}, \{1, 3\}, \{2, 4\}$ must each cross once. On the triple $(1, 2, 3)$ only $\{1, 3\}$ and $\{2, 3\}$ are inverted, so the triple rule forces $t_{23} < t_{13}$. Red is F, blue is B; circles are vias. Crossing lanes are on different layers. Lane 3 serves its two F partners in one excursion to B, and lane 4 serves one: four vias, equal to the floor $2(K - \mathrm{LIS}(\pi)) = 2(4 - 2)$ of Lemma 1.

Every lane's route is then one scalar coordinate u: on the trunk $u = s$, from the lane's *entry*, the s of its start; on ring k, $u = H_k + s' - r_k$, with s' arclength along the ring's spine and r_k the point on it past all of its handoffs. Lanes at the same u in one frame stand across the bus from one another, so a crossing's position is one number for both lanes. We discretize u on a grid G of a fifth of p, 0.0504 mm.

A pair of lanes (a, b) with $\lambda(a) < \lambda(b)$ is *inverted* if $\varphi(a) > \varphi(b)$. Every inverted pair must cross; in our model it crosses exactly once, and no other pair crosses. The set I of inverted pairs is the inversion set of the permutation π that takes launch order to berth order (Fig. 2). We call the crossings, with the order in which they occur, the *braid*; unlike a braid in topology it records no over and under, which is what the layers decide.

C. Two bounds

Lemma 1 (Floor): If both ends of every lane are on F, any routing within the frame—lanes that pass round neither array, so that they leave the source in the order λ and reach the destination in the order φ—needs at least $2(K - \mathrm{LIS}(\pi))$ vias, where LIS is the length of the longest increasing subsequence.

Proof: Lanes that never change layer stay on F and so cannot cross one another; they form an increasing subsequence of π. Every other lane leaves F and returns, at least two vias. ∎

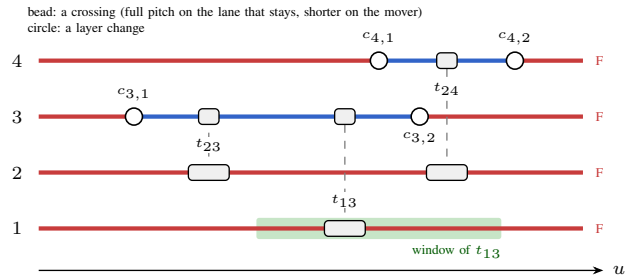


Fig. 3. The whole-route model as beads on tracks, for the four lanes of Fig. 2 (all ends on F). Each lane's route coordinate u is a track, drawn in launch order. A crossing variable t_{ab} is one bead that sits on both lanes' tracks at once (dashed tie), and intervals on one track may not overlap; each crossing slides only within its window (green, shown for t_{13}). Layer changes $c_{n,k}$ are beads too (circles), and between them a track's color is its layer. The model requires tied tracks to differ in color at their bead and each lane's number of changes to have the parity of its end layers, and the triple rule orders t_{23} before t_{13}. The solver slides every bead at once and proves the plan optimal.

The floor is the classical maximum planar subset of the two-shore case [7], and it is weak: lanes that leave F also cross one another, and an excursion to B cannot serve a crossing with a partner that is itself on B. For a uniformly random permutation the LIS grows as $2\sqrt{K}$, with Tracy–Widom fluctuations of order $K^{1/6}$ [32], the statistics of the KPZ growth class [33]; real buses are structured, but this is a yardstick for how the floor grows. The next lemma counts a lane's vias exactly, given its partners.

Lemma 2 (Blocks): Fix the layers of lane n's crossing partners at their crossings, in the order the crossings occur along n, and let both ends of n be on F. The fewest layer changes of n is twice the number of maximal runs of consecutive partners that are on F.

Proof: At each crossing n must be on the partner's opposite layer: on B across a maximal run of F partners, on F across a run of B partners and at both ends. Each F run therefore needs one change into B and one out, and no change is needed elsewhere. ∎

The difficulty is therefore not placing a lane's vias given its partners, but choosing every lane's layers and crossing order together—which is what the whole-route model does.

IV. The Whole-Route Solve

One CP-SAT [34] model decides every crossing and every layer change for all lanes at once (Fig. 3).

Crossings. For each $(a, b) \in I$ an integer $t_{ab} \in [\ell_{ab}, h_{ab}]$ on the grid G is the crossing's position along the route. Its window opens past both lanes' entries (a pair's end connector, inside which nothing may cross it) and, on two layers, past a band of 0.65 mm in front of S where the teeth stand; it closes before the earlier lane leaves the trunk, at its landing or its ring's start, or, where both ride one ring, before the earlier landing on it. The *triple rule*: for every $i < j < k$ in launch order,

- if $ij, ik, jk \in I$: $t_{ij} < t_{ik} < t_{jk}$ or $t_{jk} < t_{ik} < t_{ij}$ (one Boolean);
- if $ij, ik \in I$ and $jk \notin I$: $t_{ij} < t_{ik}$;
- if $ik, jk \in I$ and $ij \notin I$: $t_{jk} < t_{ik}$.

(The case $ij, jk \in I$, $ik \notin I$ cannot occur.) The rule is a known characterization:

Proposition 1: A linear order of I is the order in which the crossings of some wiring diagram (reduced word) of π occur if and only if it satisfies the triple rule.

Sketch: In a wiring diagram two lanes cross only when adjacent, so after any prefix of crossings the lanes stand in some permutation's order and the prefix is its inversion set; conversely, a pair whose addition keeps the prefix an inversion set is adjacent and can cross next. A set of pairs is an inversion set if and only if, on every triple, it is closed (ij, jk imply ik) and co-closed (ik implies ij or jk); requiring this of every prefix gives exactly the three cases above, Papi's characterization of convex orders [30]. ∎

Crossings of disjoint pairs may coincide—they commute—while crossings that share a lane are kept apart by a per-lane no-overlap constraint on intervals starting at each crossing: p long on the lane that *stays* and $2G$, the footprint of a steep sweep, on the lane that *moves* across, a Boolean per crossing choosing which. Two crossings of a pair lane that pass other lanes in opposite directions are kept two turning runs and a pitch apart.

Worldlines. Read u as time and the lane order as position: lanes are worldlines, and a crossing is a two-body scattering event. Three mutually crossing lanes can scatter in only two orders, $t_{ij} < t_{ik} < t_{jk}$ or its reverse—the two sides of the Yang–Baxter relation [35], or the third Reidemeister move. The triple rule's Boolean is the choice between them.

Layer changes. Each lane n has up to three change positions $c_{n,1..3}$ with activity literals $a_{n,k}$; active changes are ordered, a via's room D_v apart (0.65 mm for a single, a pitch more for a pair), and kept out of the band in front of S and out of a room at each end; their parity is fixed by the ends:

$$\bigoplus_k a_{n,k} \;=\; \ell_n^S \oplus \ell_n^D, \tag{1}$$

where $\ell_n^S, \ell_n^D \in \{0,1\}$ are the end layers. For each crossing (a, b) and each change k of lane a, a literal $\beta_{a,k,ab}$ states that the change lies before the crossing, enforced with clearance: $c_{a,k} \le t_{ab} - h$ or $c_{a,k} \ge t_{ab} + h$, where h is half of D_v, a pair's pitch more when the crossing lane is a pair, and a tenth more when a stays. The layer of a at the crossing is $\ell_a^S \oplus \bigoplus_k (a_{a,k} \wedge \beta_{a,k,ab})$, and crossing lanes must differ:

$$\ell_a^S \oplus \ell_b^S \oplus \bigoplus_k (a_{a,k} \wedge \beta_{a,k,ab}) \oplus \bigoplus_k (a_{b,k} \wedge \beta_{b,k,ab}) \;=\; 1. \tag{2}$$

Changes of lanes that can be neighbors—adjacent in λ or φ, or crossing each other—are staggered along u, within each frame, by $\max(d, \sqrt{v_v^2 - p^2})$, where d is the via's diameter and $v_v = 0.38$ mm the least distance between two vias' centers and a grid step, so that two vias one pitch p apart across the bus still meet it.

Spins. Give each worldline an Ising spin, its layer. Equation (2) is then a hard antiferromagnetic bond at every crossing, a via is a domain wall along a worldline, and the objective counts domain walls. A cycle of must-differ constraints with an odd number of links cannot be satisfied without a wall—it is frustrated [36]—and frustration is what forces vias. With the crossing order fixed this is the planar spin-glass ground state that Barahona *et al.* solved as a maximum cut alongside via minimization [11], and Lemma 2 counts the domain walls of one worldline in the field of its partners. The whole-route solve also chooses the order of the scattering events.

Built-in cuts keep a lane's changes off stretches within a via's reach of a pad of another part, or of another net's tooth, that its reference passes within a pitch of, and a pair's off its ring's corners. A cut of half-width w at u is a clause on each change: $c_{n,k} \le u - w$, $c_{n,k} \ge u + w$, or the change is not made ($a_{n,k} = 0$).

Objective. With $o_n \ge 0$ and $o_n \ge s_n + \sum_k a_{n,k} - 2$ counting the vias of lane n beyond two (s_n its stub vias, the larger leg's for a pair, less a tie via that joins a ball to a pad of its own net under it),

$$\min\; W_O \sum_n o_n \;+\; W_V \sum_{n,k} a_{n,k} \;+\; W_S \sum_j b_j \;+\; H, \tag{3}$$

with $W_V = 10^6$, $W_O = 5W_V$, $W_S = 3W_V$ for each soft cut j broken (Section VI-E), and H the history term of Section VI-E, a small fraction of a via for each place it prices. Preferring at most two vias per net is a price, not a cap: a cap is infeasible on many instances, and a lexicographic preference would buy one net's third via back with any number of other vias.

Proving it. CP-SAT runs on four workers in deterministic interleaved batches, clause sharing off, so every run is the same. A first *probe* restricts its full-problem searches to those that raise the bound—core-based search and two linear relaxations—for at most ten batches; CP-SAT's default portfolio dilutes these among five plan-finding searches, and left the 51-net rung's first solve unproved where the probe proves it. If the probe's plan is not proved in whole vias, plan-finding searches continue from it, with core-based search and the probe's bound as a constraint, for at most a hundred batches. A re-solve in the loop starts from the previous plan and only adds cuts and history, so a proved first solve's via terms are a valid floor for it, and a plan at that floor is proved at once. A plan the search cannot prove is still kept and laid, its nets over two vias returned to the ends; where there is no plan at all, a relaxed model names the crowded lanes, and they return to the ends. On both ladders every first solve was proved (Section VII-G).

Scope. The proof is relative to the model: the ends are fixed, each inverted pair crosses once (non-reduced braids are excluded), a lane changes layer at most three times (no plan on either ladder uses more than two), positions lie on G, and the built-in and loop cuts are heuristic. What is proved is the least objective in whole vias for the given ends: the layer changes, five more per via a net carries over two, and three per soft cut broken. The board's via count also depends on the ends, whose stub vias are about half of it, and a better choice of ends can beat a proved plan.

V. Choosing the Ends

Because the ends decide the braid, the escapes at both arrays are chosen with an estimate of the solve's objective, to which the search adds prices of its own: the lanes' length at a fixed rate per via, trunk congestion and crossings, and the audits' feedback (Section VI-E). For a candidate assignment of teeth

and berths we estimate the route's vias from the ends alone: among lanes whose ends share a layer, inverted pairs on one layer must be separated by lanes that dive, and since their conflict graph is a permutation graph the cheapest cover is the complement of a maximum-weight increasing chain, exact per layer; a lane whose ends differ changes once, and the partners it would meet on the wrong layer form a bipartite graph covered exactly by a minimum cut; conflicts between such lanes are resolved greedily. Congestion adds each lane's load on the trunk (the room its crossings and changes take), priced by the square of its excess over half, and a fifth of a via per trunk crossing. The crossing price exists because the solve's effort follows its crossings, not its vias. The search moves one tooth or berth at a time (ejecting one conflicting lane where needed), then iterates from seeded random kicks. The best few states are then re-ranked *exactly* on their orders by a lighter form of the solve's model, with no crossing positions: a Boolean per triple of mutually crossing lanes, one transitive order of each lane's crossings, and each layer change before or after each of them. Since of c mutually crossing lanes on two layers at least $c - 2$ change layer, that bound is added for every maximal clique of the crossing graph, and CP-SAT searches core-guided, since nearly every lane has no change. Each exact ranking stops at a work limit and the search is local, so the chosen ends are not proved best.

Street berths. A ball beside the unpopulated band of a DDR3 package may dive in a lane of the band and leave along it on the other layer, as the human designers do. Offering such berths trades a few vias for fewer crossings, and so for a quicker solve.

VI. From Plan to Copper

A plan becomes copper in stages, each with a margin over the next: the geometry and the polish keep a grid step more room than the snap and the audit. The geometry and the polish place the lanes in millimeters, and an audit gates this smooth plan; the snap then lays the pairs on the router's grid, the polish fits the singles round them, and the snap lays the singles, each step audited; a last audit, with a lint of the router's shape rules, gates the snapped plan, and the production router lays it.

A. Geometry

A single linear program places every lane: an offset per lane per column, columns every 0.1 mm along the trunk and both rings, a ring lane's two pieces joined where it leaves the trunk. The solve fixes each column's order and each lane's layers; the program moves lanes and vias across the spine only. Constraints keep same-layer neighbors a pitch apart, a via's room round every change (chords of a disc at neighboring columns), every lane inside the board and off the pad boxes, and the slope's change within one per column, a 45° turn from a run along the spine; the ends are pinned, with a few columns beside them held parallel to the spine. Costs are the added length—under-estimated by tangent cuts, so that a slope below 13° costs nothing and lanes take a *comfortable* pitch where there is room—the bends, and a hinge penalty on neighbors closer than the comfortable pitch $2p$. Every hard constraint except the pins, the holds and the joins is elastic, each row's slack priced at $W_H = 10^4$ per millimeter, so the program always has a solution and reports exactly where it had to break a rule.

A second pass holds each lane to one side of every *island*—a part's pads, joined with any pads closer than a lane can pass between. The choice of side is non-convex, so it is made combinatorially before the pass, by neither the LP nor the CP-SAT model: at each island the lanes, in the order the CP-SAT solve fixed there, are split at the index that the first pass's offsets make cheapest, a split leaving either side too little room for its lanes priced out, and the island is inserted into the order at that split. The sides are thus chosen consistent with the crossing plan, and the program stays convex; the polish may still flip a lane it cannot hold on its side (Section VI-B). The second pass also corrects each pitch for slope by tangent cuts at the first pass's slopes. Most of its rows are such cuts and few bind, so they are added lazily: the pass is solved with those within 0.1 mm of binding at the first pass's answer, then again with those near binding at any answer that breaks one, each solve warm-started from the last one's basis, until none is broken. The result is the whole program's optimum. The constraints paid in the second pass become cuts for the solve: a lane that could not be kept off an island may not cross anyone over the island's span (an *island cut*); a change that could not be given its room may not lie there (a *via cut*).

The program is large—on the 51-net rung 7 228 lane offsets and 150 753 rows in pass one, and 246 648 rows in pass two, of which its four lazy solves use 148 257—so we pass its dual to HiGHS [37] through SciPy, reading the offsets back as the multipliers of the dual's balance equations; the two passes take 26 and 124 s.

Forces. The program's optimality conditions are those of a mechanism at rest (Fig. 4). Each lane point is a bead that slides on its column's rail, the length term is a string—slack near the spine, its tension rising toward unity as it leans—and the rules are contacts: struts between neighbors, walls at the pads and the board edge. A rule's dual value is the force in its contact, and complementary slackness is the contact law—a strut pushes only while it touches. An elastic row is a rigid-plastic strut that carries up to W_H and then yields. The static and kinematic theorems of plastic collapse are this duality [38], and the linear program that tests a packing of hard spheres for jamming solves for the same contact forces [39]. Only a small fraction of the constraints carry a force, and they name the rules that hold each lane where it is; a constraint whose slack is paid is a strut that yields, and those of the static, via and dive kinds return to the solve as cuts.

B. Polish and snap

The *polish* meets the audit's own measures in board coordinates—pitch, via room, clearance to pads as drawn, stub joins and turn shapes—by small vertex moves, one linear program per round within a trust box of ± 0.05 mm, every rule elastic. A rule enters a round's program while it stands within 0.1 mm of its bar; a lane or via found inside a pad is pushed out of the pad's island, not toward its nearest edge; folds and

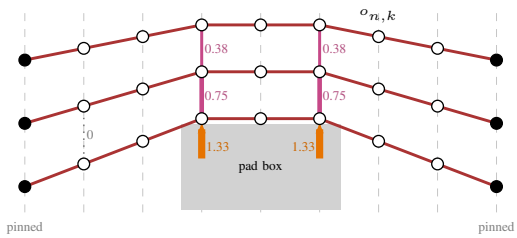


Fig. 4. A small instance of the geometry LP, solved: three lanes on one layer, pinned at both ends (black), pass over a pad box. Each lane point slides on its column's rail (dashed); the lanes are strings at unit tension (the toy also pulls each weakly toward its straight reference). The binding rules carry the dual values drawn: the box (orange) pushes the lowest lane up where the lanes bend, and pitch struts (magenta) pass the load up the stack. Every other rule, like the strut near the pins (dotted), carries no force. Forces are in units of the string tension.

notches the audit would flag are straightened between cycles of rounds; and the state kept is the best any round reached, not the last. A pair's end runs, and the straight runs the pair router needs either side of each dive, are laid straight first and held. A lane that cannot be held on its side of an island is flipped, and the geometry runs again; a via that cannot be given its room becomes a via cut.

The *snap* makes the smooth plan octilinear on the router's grid, one lane at a time, by an A* search in a strip $2p$ either side of each lane's smooth line that prices length, bends and distance from the line, each via within $2p$ of the plan's. Cells are blocked as the router will block them: by the router's own map of static copper, by the real copper of lanes already laid, and by the *shares* of lanes not yet laid—each keeps the side of every midline nearer its own line, less half a bar—so lanes laid first cannot take the room later ones need. Pairs go first, moving exactly as the pair router moves ($45°$ turns, a straight run after each turn, a via only on a straight run); they are then held while the polish fits the singles round them, and the singles are snapped. Up to two sweeps then lay every lane again against the others' real copper. A pair that cannot be laid returns a via cut at its nearest change.

C. Differential pairs

A pair is one lane of two slots' width. Its *hand* at an end is the side of its travel on which P lies. A pair whose hands at its tooth and berth disagree must swap its legs somewhere; the solve then requires it to change layer at least once, and the snap lays a *crossover* at its first dive, as a designer would: the first leg steps out at $45°$ to its barrel, dives, and jogs across the pair's line on the new layer; the second jogs at $45°$ over it to a barrel just beyond, dives, and steps onto the first leg's old line. Both barrels lie on one side of the pair, staggered by the fewest whole grid steps that keep a via's pitch and each jog's clearance, and every stage knows the crossover's footprint before it is laid. At each end the legs run, through an *end connector*—two legs from the pair's tips, turning at most $45°$ at their knees and onto the pose's heading—onto a *pose*, a grid point and heading from which the pair router takes over, so plan and router share one end.

D. Audit and route

The audit checks the plan against the router's rules as the router will see them (pads as drawn, each terminal joined as the router joins it), waiving only shortfalls within a quarter grid step, which the router's own rounding moves; the snapped plan must also pass a lint of the router's shapes (vertices on the grid, $45°$ angles, a via at every layer change, the pair router's turns and straights). The gate passes a plan only when it is complete and every count is zero. The production router [1] then lays the pairs, then the singles, each lane confined to its *band*—the router cells within half a grid step of its snapped line, with its end and via cells. A pair the router refuses in its band is tried in wider ones, and free of the plan last, after every lane has had its band, so that it cannot take the room the plan gave the singles. The audit models the router's rules but is not the router: a plan that passes the gate can still have a lane refused, which is why the board, not the plan, is graded. Independent checks grade connectivity and DRC on the written board.

E. The negotiation loop

Every stage is fed a measurement of the one before (Fig. 5); a round ends at the first stage that fails, with what that stage found. Every audit's findings return to the solve as *history*: every place a round found the plan short is priced, as a negotiated router prices a place overused before [29], by how many audits found it so: about 10^{-4} of a via for a crossing there and 10^{-3} for a layer change, for each audit. History thus reorders plans of equal vias; the solve's proof assumes its total stays below a via. The geometry's and the polish's paid constraints and the pair snap's failures return as cuts, and a finding at a part on one layer only as a soft *layer cut* that holds the lane on the other layer there; side flips return to the geometry. When a re-solve proves no plan under its hard cuts, every hard cut so far becomes a soft cost of three vias. The loop stops when the gate passes, after six rounds or two that do not improve on the best, or when a finding stands at the ends, which only the fanout can move. After a loop that does not pass, its findings at the ends are priced in the ends model, doubling when a later round names the same end again, and the next fanout round chooses again, freeing only the ends named so far.

Never nothing. A loop that does not pass is still laid, from its last snapped plan or else its best round's, and a lane that cannot be laid stays open; a round that lays connected and DRC-clean ends the run, whether or not its loop passed and its solve proved the plan. A round that leaves nets open sends them back to the fanout, and the run's result is its best round: no DRC violation first, then the fewest open nets, then vias. This is a counterexample-guided loop in the spirit of combinatorial Benders decomposition [40], [41]; unlike it, a cut here generalizes one geometric failure and is not proved valid, so the loop's result is optimal only for the model it ends with.

Sectors. The geometry's non-convexity lives entirely in discrete choices made before it—the crossing order, the layers and the side of each island, together a homotopy class of the whole bus. Within one such topological sector, and with the

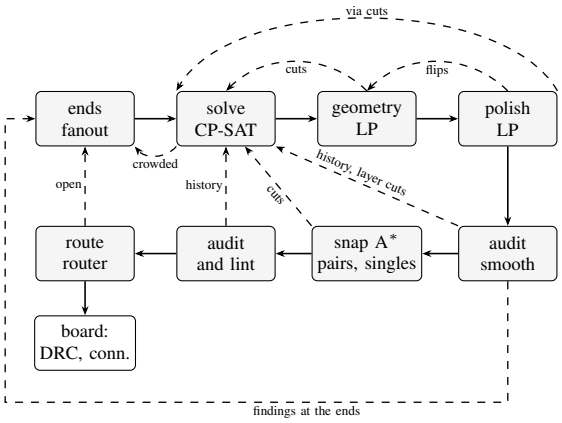


Fig. 5. The whole route. Solid arrows pass the plan forward: an audit gates the smooth plan before the snap, and a last audit with a lint gates the snapped plan before the router lays it. Dashed arrows are the negotiation: lanes with no room beside an island return to the geometry as side flips; rules the geometry or the polish had to break, and lanes the snap could not lay, return to the solve as cuts; the audits' findings return as history, and those at a one-layer part as layer cuts; findings at the ends, open nets and crowded lanes return to the choice of ends.

first pass's linearization fixed, the geometry is convex. The loop is a search over sectors: the solve proposes one, the geometry relaxes within it, and the constraints that yield tell the solve which sectors to leave.

VII. Results

A. Benches and grading

H3. An Allwinner H3 (0.65 mm ball pitch) to a DDR3 device on a public reference design [42], whose designer routed the bus on its two layers. Its bench carries 51 bus nets, among them three differential pairs (two data strobes and the clock); the ladder's rungs K15–K51 add whole groups of nets, tightest first. *Zynq.* The Zynq-7020 (CLG400) and its DDR3 on a public design [43], whose designer routed the bus on the outer layers only (the inner layers are planes), every via F to B. We strip its routing, fan out the processor and keep the two outer layers, by a recipe from the board file in the repository [1]; the ladder's rungs K18–K43 carry both strobe pairs.

Each result is written as a board and graded independently: connectivity of every bus net, DRC at the board's clearance of 0.1 mm, and a census of every via and millimeter of the run's nets, stubs included. The other routers' boards and the human's are graded the same way. Runs used CP-SAT from OR-Tools 9.15 and HiGHS through SciPy 1.16, on Python 3.14, on an Apple M3 laptop (8 cores, 8 GB), at the toolkit's commit `2e319092`.

B. Against the human

Table I gives every rung of both ladders on our own ends. Every rung routes every lane in its band at once, every net connected, DRC-clean, never with more vias than the human and always with less copper; every rung but H3 K28 and K35, whose first fanout the fanout's own audit refused, does so in its first fanout round. At the top of the H3 ladder the route uses 80 vias to the human's 88 (Fig. 6); on the Zynq, 62 to 100. Part of the copper difference is the human's length matching.

C. Against other routers

Two other routers ran on the same benches, nets and design rules, graded the same way (Table I): the production chain of the toolkit the whole route belongs to [1], without the bus step—its BGA fanout at the destination, its differential-pair router and its negotiated A* router—and Freerouting 2.4.1 [44], an open-source autorouter, through KiCad's Specctra export, with its default settings but two: at most 100 passes, which no run reached, and its score's charge for a clearance violation, a fifth of an unrouted connection, lowered a thousandfold. Its own check counts the benches' fixed copper, legal at the board's clearance, as up to 106 violations, mostly where its octagons stand out past round pads and track ends; at the default charge these held the smaller Zynq rungs' score at zero and stopped them early. Freerouting routes each leg of a differential pair as an independent net.

The production chain completes six of the ten rungs and leaves one to four nets open on the others, with 2.0 to 4.7 times the whole route's vias (319 against 80 at H3 K51) and more copper on every rung; at Zynq K43 its board also has seven clearance violations. Freerouting completes none: it leaves 1 to 5 nets open, and at Zynq K33 its board also has seven DRC violations. On the nets it does connect it can use fewer vias than the whole route—44 against 54 at H3 K41, with four nets open—because its lanes enter the destination's ball field from the near face, where our ends escape every ball to the array's edge (Section VIII). In wall time on the same laptop, the production chain takes 5 to 59 s a rung and Freerouting 1.6 to 3.9 minutes; the whole route takes 18 to 290 s, 3.6 to 6.8 times the production chain's time and from a fifth of Freerouting's to 1.5 times it. On the Zynq bus on its own board (Section VII-D) the bus step takes 382 s, the production chain 78 s and Freerouting 164 s.

D. On the board itself

The Zynq bench cuts the bus out of its board at the paper's uniform sizes. The toolkit's *bus step* instead routes it on the board itself, as the production chain hands the board on: planes poured on the inner layers, the board's own design rules, both arrays fanned out by the step, and the lanes on F and B, where the designer routed them. The step takes 46 of the nets between the arrays, leaving the DDR reset's pull-down stub to the rest of the chain; it refuses the clock pair, which passes through a series termination, and the column-address strobe, for which its fanout found no escape. The production chain and Freerouting ran on the same board, nets, layers and rules, and every board, the human's included, was graded on the 45 nets other than the reset at the board's clearance (Table I, Fig. 7). The whole route completes them in its first round, with 70 vias to the human's 105 and less copper; the production chain leaves one net open and two clearance violations, with 162 vias, and Freerouting two nets open with 70.

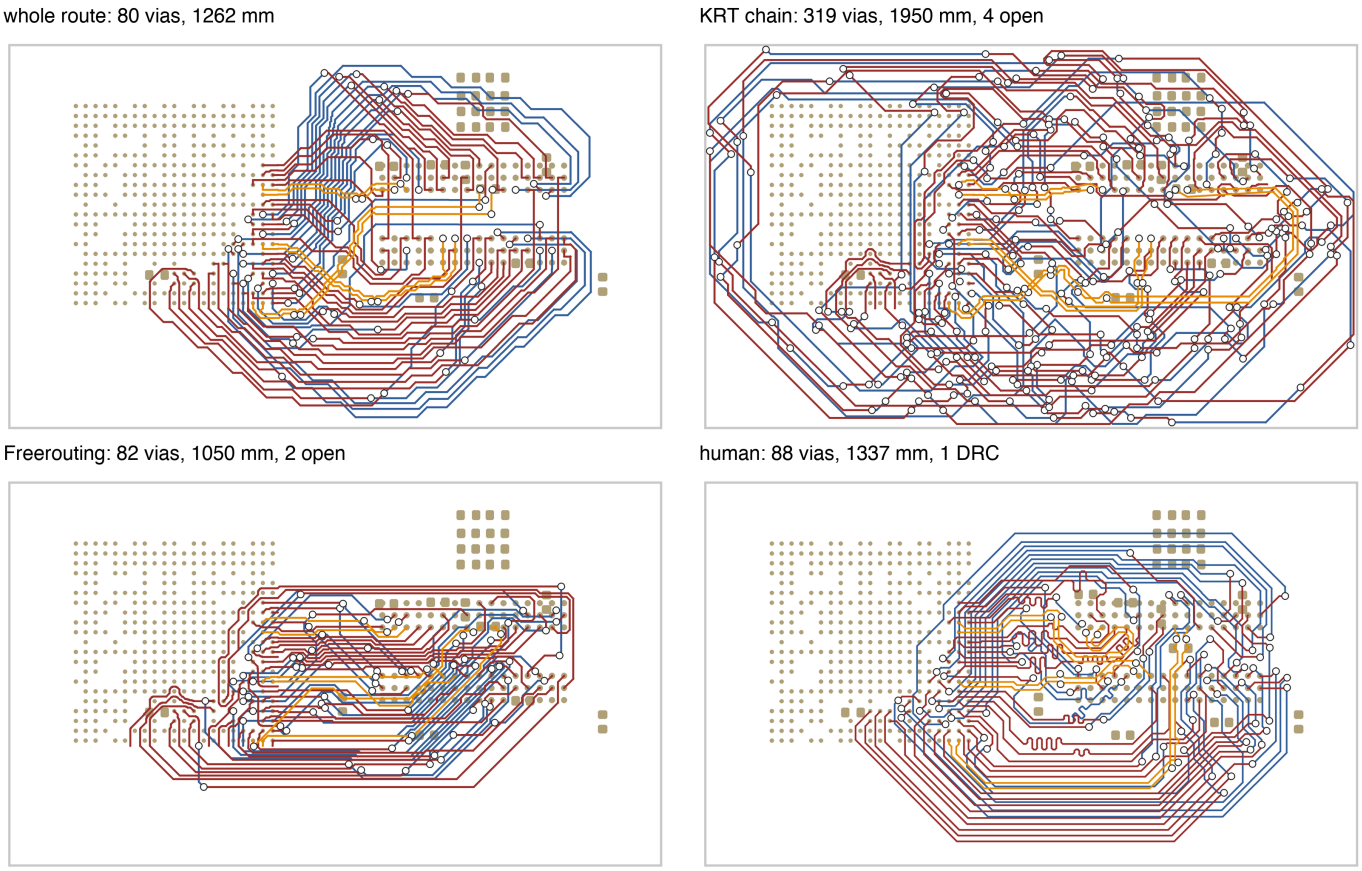


Fig. 6. The 51-net H3 rung by the whole route on its own ends, the toolkit's production router without the bus step, Freerouting, and the human, only the run's nets drawn; each panel is labeled with its vias, copper, open nets and DRC violations. F is red, B blue, the three differential pairs orange; every via is drawn as the same marker. The source BGA is at left, the DDR3 at right.

TABLE I
VIAS, COPPER (MM), OPEN NETS AND DRC VIOLATIONS OF THE RUN'S NETS ON THE FINISHED BOARD: THE WHOLE ROUTE, THE TOOLKIT'S PRODUCTION ROUTER WITHOUT THE BUS STEP (KRT CHAIN), FREEROUTING 2.4.1, AND THE HUMAN'S BOARD, EVERY BOARD GRADED THE SAME WAY. THE ZYNQ'S LAST ROW, FULL, IS ITS BUS ON ITS OWN BOARD (SECTION VII-D), ON THE 45 NETS GRADED THERE.

	whole route				KRT chain				Freerouting				human			
rung	vias	mm	open	DRC	vias	mm	open	DRC	vias	mm	open	DRC	vias	mm	open	DRC
Allwinner H3 to DDR3																
K15	12	194	0	0	40	424	0	0	16	153	1	0	22	232	0	0
K28	32	561	0	0	72	877	0	0	29	416	5	0	48	678	0	1
K35	44	758	0	0	138	1340	0	0	41	631	3	0	60	889	0	1
K41	54	967	0	0	254	1749	2	0	44	752	4	0	70	1081	0	1
K51	80	1262	0	0	319	1950	4	0	82	1050	2	0	88	1337	0	1
Zynq-7020 to DDR3																
K18	18	480	0	0	36	562	0	0	20	395	2	0	45	653	0	0
K26	28	761	0	0	72	800	0	0	36	620	2	0	57	915	0	0
K33	44	990	0	0	104	1131	0	0	50	851	2	7	77	1197	0	0
K38	56	1100	0	0	128	1290	1	0	58	1007	2	0	87	1371	0	0
K43	62	1299	0	0	151	1601	1	7	64	1087	4	0	100	1577	0	0
Full	70	1457	0	0	162	1622	1	2	70	1247	2	0	105	1658	0	0

E. Synthetic buses

The repository's generator plants a crossing pattern between two arrays and computes the optimum of its *channel*—every tooth on the source's facing column and every berth on the destination's, in the balls' order—exactly, over every crossing order [1]. Twenty-nine such buses of 4 to 16 nets span sorted, blocked, interleaved, shuffled and reversed patterns, short 3 mm channels, all-via escapes, walls of parts in the channel, rings round the destination, pairs, and lanes that must wind round it. On two layers the whole route completes 23 of them connected and DRC-clean. Of the 16 whose channel optimum applies, it lays exactly the optimum on 10 and completes two more above it, at 18 and 34 vias where 10 and 14 are optimal, with every escape a via. The six it leaves incomplete all have

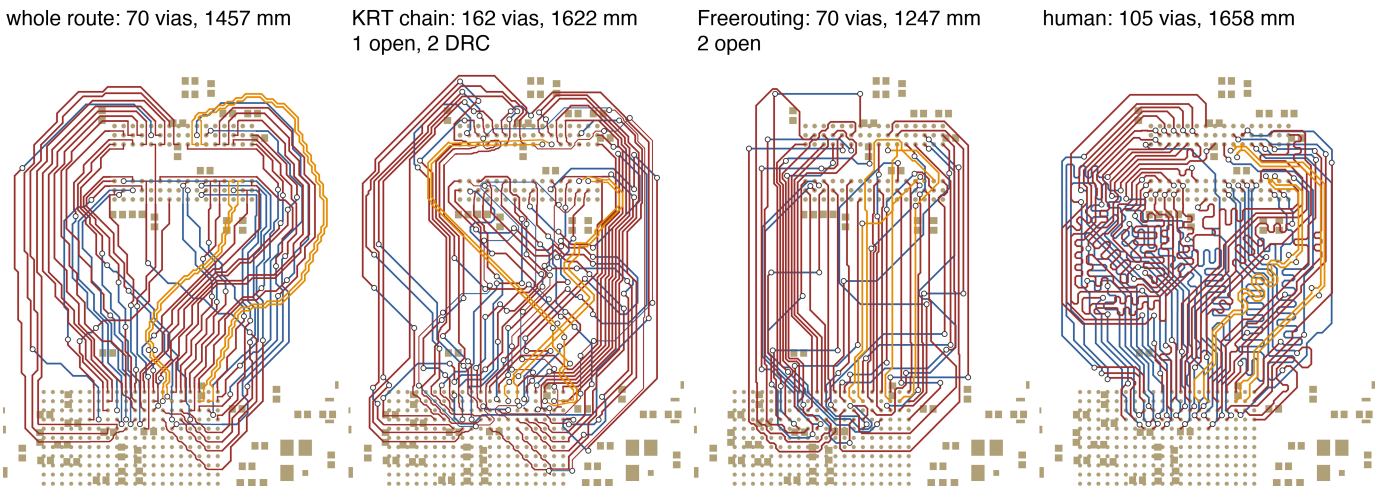


Fig. 7. The Zynq DDR3 bus on the board itself, drawn as Fig. 6: the whole route's bus step, the toolkit's production router, Freerouting and the human, the 45 nets graded on each, each panel labeled with its vias, copper, open nets and DRC violations. The processor is at the bottom, the DDR3 at the top.

the short channel, and leave one to five nets open: two layers do not give the changes room there. On cases without a graded optimum the route can lay fewer vias than the channel's floor, because it chooses its own ends and may leave an array by another face.

F. More routing layers

The model extends to $q > 2$ routing layers with few changes. A lane's layer between its changes takes one of q values, two lanes cross only on different layers, and a crossing's room is shared only by the lanes that cross on one layer. An end whose stub carries a via may leave on any layer its run fits, the solve choosing which, and the run is moved onto that layer before the geometry. In the spin picture the layer becomes a q-state Potts spin [45] with an antiferromagnetic bond at every crossing, and frustration begins only at $q+1$ mutually crossing lanes: the clique bound of Section V becomes $c-q$ changes among c such lanes. On the 29 generated buses on three, four and six routing layers, all 87 runs complete connected and DRC-clean, and all 48 whose channel optimum applies lay exactly that optimum, the short channels and all-via escapes included. More layers do not lower these channels' optimum, which equals the floor of Lemma 1 on every layer count since the arrays' pads are on F; what they give is room for the changes.

G. Cost

Of H3 K51's 290 s, the fanout with its exact re-ranking takes 42 s, the solve 32 s, the geometry LP 150 s, the polishes 19 s, the snaps 27 s, the audits and the lint 11 s, and laying the plan 6 s; the geometry is about half the time at the top of both ladders. The time grows a little faster than the bus, from 1.2 s a net on the 15-net H3 rung to 5.7 s on the 51-net one, and peak memory is 250 to 720 MB. On the Zynq board the whole route takes 357 s of the bus step's 382, 119 of them fanning out both arrays and 122 the geometry. The whole route runs mostly on one core, its CPU time within 11% of its wall time on every rung, so running three rungs and the board's bus step side by side cost it little waiting. Every rung's first solve is proved optimal: at the top of the H3 ladder its 313 crossings and 2 744 triples in 30 s, every other rung's in 7 s or less, and on the board 229 crossings and 1 922 triples in 46 s.

VIII. Limitations and Future Work

The route does not match lengths; a DDR interface requires length (or delay) matching within byte lanes and of the address group to the clock [46]. Matching lengths is our next step: as recent length matching assigns each trace a region by linear programming [47], [48], each lane's meander area can become a variable of the geometry LP. On more than two routing layers the model is so far graded only on generated buses (Section VII-F), and a pair through a series termination between the arrays is left to the rest of the chain, since the whole route lays no waypoint. Optimality is relative to the model, and the ends are chosen by a local search whose estimate is exact only in its parts; a model over the ends that bounds the route's own objective could prove them. Ends inside the destination's ball field, where Freerouting's lanes enter it, are a natural extension. Letting the ends search swap a DDR controller's data bits within a byte lane, as pin assignment has been chosen jointly with escape routing [49], could remove most crossings within it. With the crossing order fixed and via room set aside, the layer assignment is a planar Ising ground state, solvable in polynomial time [10], [50], [51]; it could give the ends search a fast exact evaluator and a lower bound. The solve counts a pair's dive once where the board counts two barrels, and intra-pair skew is not yet planned. Finally, two boards are few, and the design constants were developed on them; we are extending the evaluation to more boards.

IX. Conclusion

Planning a bus whole—ends, crossings, their order and every layer change decided at once, the plan proved optimal for the chosen ends and negotiated with the geometry until

the production router can lay it—routes one real DDR3 bus whole and 43 nets of another fully automatically, and 45 of the second's nets on its own board, with fewer vias and less copper than their designers. The physics proved of practical use: the worldline picture gives the triple rule its meaning, the spin picture traces every via to frustration, and the forces of the linear program's dual name the rules that hold each lane and the ones that must yield. We expect the same pairing—an exact choice of topological sector, relaxed in a convex statics problem whose forces steer the next choice—to carry over to length matching and to boards with more layers.

REFERENCES

[1] A. Haas, "KiCadRoutingTools," [Online]. Available: https://github.com/drandyhaas/KiCadRoutingTools, 2026, commit `2e319092`.

[2] R. S. Lawson, B. A. Neal, D. K. Morgan, and J. Radumilo-Franklin, "System and method for electronic design routing between terminals," U.S. Patent 9 202 001, Dec. 1, 2015.

[3] C. Pfeil, "What's the difference: PCB routing now and then," Electronic Design, [Online]. Available: https://www.electronicdesign.com/resources/whats-the-difference-between/article/21800231/whats-the-difference-pcb-routing-then-and-now, Sep. 2014.

[4] M. M. Ozdal and M. D. F. Wong, "Algorithms for simultaneous escape routing and layer assignment of dense PCBs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 25, no. 8, pp. 1510–1522, 2006.

[5] C.-P. Hsu, "Minimum-via topological routing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 2, no. 4, pp. 235–246, 1983.

[6] M. Marek-Sadowska, "An unconstrained topological via minimization problem for two-layer routing," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 3, no. 3, pp. 184–190, 1984.

[7] M. Sarrafzadeh and D. T. Lee, "A new approach to topological via minimization," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 8, no. 8, pp. 890–900, 1989.

[8] R. Y. Pinter, "Optimal layer assignment for interconnect," *J. VLSI Comput. Syst.*, vol. 1, no. 2, pp. 123–137, 1984.

[9] N. J. Naclerio, S. Masuda, and K. Nakajima, "The via minimization problem is NP-complete," *IEEE Trans. Comput.*, vol. 38, no. 11, pp. 1604–1608, 1989.

[10] F. O. Hadlock, "Finding a maximum cut of a planar graph in polynomial time," *SIAM J. Comput.*, vol. 4, no. 3, pp. 221–225, 1975.

[11] F. Barahona, M. Grötschel, M. Jünger, and G. Reinelt, "An application of combinatorial optimization to statistical physics and circuit layout design," *Oper. Res.*, vol. 36, no. 3, pp. 493–513, 1988.

[12] C. S. Rim, T. Kashiwabara, and K. Nakajima, "Exact algorithms for multilayer topological via minimization," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 8, no. 11, pp. 1165–1173, 1989.

[13] J. Cong and C. L. Liu, "On the k-layer planar subset and topological via minimization problems," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 10, no. 8, pp. 972–981, 1991.

[14] T.-C. Lin, C.-C. Chi, and Y.-W. Chang, "Redistribution layer routing for wafer-level integrated fan-out package-on-packages," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2017, pp. 561–568.

[15] M. M. Ozdal, M. D. F. Wong, and P. S. Honsinger, "Simultaneous escape-routing algorithms for via minimization of high-speed boards," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 27, no. 1, pp. 84–95, 2008.

[16] H. Kong, T. Yan, and M. D. F. Wong, "Automatic bus planner for dense PCBs," in *Proc. 46th ACM/IEEE Design Automat. Conf. (DAC)*, 2009, pp. 326–331.

[17] J.-T. Yan, "Layer assignment of buses and nets with via-count constraint in high-speed PCB designs," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 38, no. 3, pp. 512–525, 2019.

[18] T. Yan and M. D. F. Wong, "Untangling twisted nets for bus routing," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2007, pp. 396–400.

[19] ——, "Theories and algorithms on single-detour routing for untangling twisted bus," *ACM Trans. Design Autom. Electron. Syst.*, vol. 14, no. 3, pp. 1–21, 2009.

[20] L. Luo and M. D. F. Wong, "Ordered escape routing based on Boolean satisfiability," in *Proc. Asia South Pacific Design Automat. Conf. (ASP-DAC)*, 2008, pp. 244–249.

[21] S. Bayless, H. H. Hoos, and A. J. Hu, "Scalable, high-quality, SAT-based multi-layer escape routing," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2016, pp. 1–8.

[22] Y. Tomioka, Y. Kurata, Y. Kohira, and A. Takahashi, "MILP-based efficient routing method with restricted route structure for 2-layer ball grid array packages," *IEICE Trans. Fundamentals Electron., Commun. Comput. Sci.*, vol. E92-A, no. 12, pp. 2998–3006, 2009.

[23] T. Yan, P.-C. Wu, Q. Ma, and M. D. F. Wong, "On the escape routing of differential pairs," in *Proc. IEEE/ACM Int. Conf. Comput.-Aided Design (ICCAD)*, 2010, pp. 614–620.

[24] H. Wu, X. Jiang, Y. Fan, X. He, and M. Liu, "Topology optimization-based layer assignment method for fixed-layer complex PCBs," *IEEE Trans. Compon., Packag., Manuf. Technol.*, vol. 16, no. 1, pp. 20–31, 2026.

[25] S.-T. Lin, H.-H. Wang, C.-Y. Kuo, Y. Chen, and Y.-L. Li, "A complete PCB routing methodology with concurrent hierarchical routing," in *Proc. 58th ACM/IEEE Design Automat. Conf. (DAC)*, 2021, pp. 1141–1146.

[26] W. W.-M. Dai, T. Dayan, and D. Staepelaere, "Topological routing in SURF: Generating a rubber-band sketch," in *Proc. 28th ACM/IEEE Design Automat. Conf. (DAC)*, 1991, pp. 39–44.

[27] C. E. Leiserson and F. M. Maley, "Algorithms for routing and testing routability of planar VLSI layouts," in *Proc. 17th ACM Symp. Theory Comput. (STOC)*, 1985, pp. 69–78.

[28] H.-T. Wen, Y.-J. Cai, Y. Hsu, and Y.-W. Chang, "Via-based redistribution layer routing for InFO packages with irregular pad structures," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 41, no. 12, pp. 5554–5567, 2022.

[29] L. McMurchie and C. Ebeling, "PathFinder: A negotiation-based performance-driven router for FPGAs," in *Proc. ACM Int. Symp. Field-Programmable Gate Arrays (FPGA)*, 1995, pp. 111–117.

[30] P. Papi, "A characterization of a special ordering in a root system," *Proc. Amer. Math. Soc.*, vol. 120, no. 3, pp. 661–665, 1994.

[31] Y. I. Manin and V. V. Schechtman, "Arrangements of hyperplanes, higher braid groups and higher Bruhat orders," in *Algebraic Number Theory – in honor of K. Iwasawa*, ser. Adv. Stud. Pure Math., 1989, vol. 17, pp. 289–308.

[32] J. Baik, P. Deift, and K. Johansson, "On the distribution of the length of the longest increasing subsequence of random permutations," *J. Amer. Math. Soc.*, vol. 12, no. 4, pp. 1119–1178, 1999.

[33] M. Prähofer and H. Spohn, "Universal distributions for growth processes in 1+1 dimensions and random matrices," *Phys. Rev. Lett.*, vol. 84, no. 21, pp. 4882–4885, 2000.

[34] L. Perron and F. Didier, "CP-SAT," Google OR-Tools, v9.15. [Online]. Available: https://developers.google.com/optimization/cp/cp_solver/, 2025.

[35] R. J. Baxter, "Partition function of the eight-vertex lattice model," *Ann. Phys.*, vol. 70, no. 1, pp. 193–228, 1972.

[36] G. Toulouse, "Theory of the frustration effect in spin glasses: I," *Commun. Phys.*, vol. 2, pp. 115–119, 1977.

[37] Q. Huangfu and J. A. J. Hall, "Parallelizing the dual revised simplex method," *Math. Program. Comput.*, vol. 10, no. 1, pp. 119–142, 2018.

[38] A. Charnes, C. E. Lemke, and O. C. Zienkiewicz, "Virtual work, linear programming and plastic limit analysis," *Proc. R. Soc. Lond. A*, vol. 251, no. 1264, pp. 110–116, 1959.

[39] A. Donev, S. Torquato, F. H. Stillinger, and R. Connelly, "A linear programming algorithm to test for jamming in hard-sphere packings," *J. Comput. Phys.*, vol. 197, no. 1, pp. 139–166, 2004.

[40] J. N. Hooker and G. Ottosson, "Logic-based Benders decomposition," *Math. Program.*, vol. 96, no. 1, pp. 33–60, 2003.

[41] G. Codato and M. Fischetti, "Combinatorial Benders' cuts for mixed-integer linear programming," *Oper. Res.*, vol. 54, no. 4, pp. 756–766, 2006.

[42] hvwyl, "allwinner-h3-ddr3-16x1-kicad: Allwinner H3 with DDR3 16x1 design reference for KiCad," [Online]. Available: https://github.com/hvwyl/allwinner-h3-ddr3-16x1-kicad, 2023.

[43] kangyuzhe666, "ZYNQ7010-7020_AD9363: Open-source SDR hardware based on Zynq and AD9363," [Online]. Available: https://github.com/kangyuzhe666/ZYNQ7010-7020_AD9363, 2020.

[44] Freerouting contributors, "Freerouting," [Online]. Available: https://github.com/freerouting/freerouting, 2026, v2.4.1.

[45] F. Y. Wu, "The Potts model," *Rev. Mod. Phys.*, vol. 54, no. 1, pp. 235–268, 1982.

[46] M. M. Ozdal and M. D. F. Wong, "A length-matching routing algorithm for high-performance printed circuit boards," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 25, no. 12, pp. 2784–2794, 2006.

[47] J. Lin, L. Guo, and W. Fang, "LP-based area assignment for length-matching routing of complex multilayer PCBs with any-direction wires," *ACM Trans. Design Autom. Electron. Syst.*, vol. 31, no. 5, pp. 1–22, 2026.
[48] W. Fang, L. Guo, J. Lin, S. Xiong, H. He, J. Xu, and J. Chen, "Obstacle-aware length-matching routing for any-direction traces in printed circuit board," in *Proc. 61st ACM/IEEE Design Automat. Conf. (DAC)*, 2024, pp. 1–6.
[49] S.-I. Lei and W.-K. Mak, "Simultaneous constrained pin assignment and escape routing considering differential pairs for FPGA-PCB co-design," *IEEE Trans. Comput.-Aided Design Integr. Circuits Syst.*, vol. 32, no. 12, pp. 1866–1878, 2013.
[50] F. Barahona, "On the computational complexity of Ising spin glass models," *J. Phys. A: Math. Gen.*, vol. 15, no. 10, pp. 3241–3253, 1982.
[51] M. Glass and M. Feder, "Maximum cut algorithms and upper bounds for planar and toroidal graphs," arXiv:2606.28478, 2026.